01 How the game fits together

Battle system architecture guide

A code walkthrough, runtime relationship map, and current product requirements snapshot. Prepared for the project owner on November 9, 2026.

The core idea: every character uses the same battle machinery. The character supplies its own stats and list of actions. A shared encounter controller asks the grid and combat rules what is legal, changes the battle state, then tells the presentation to show the result.

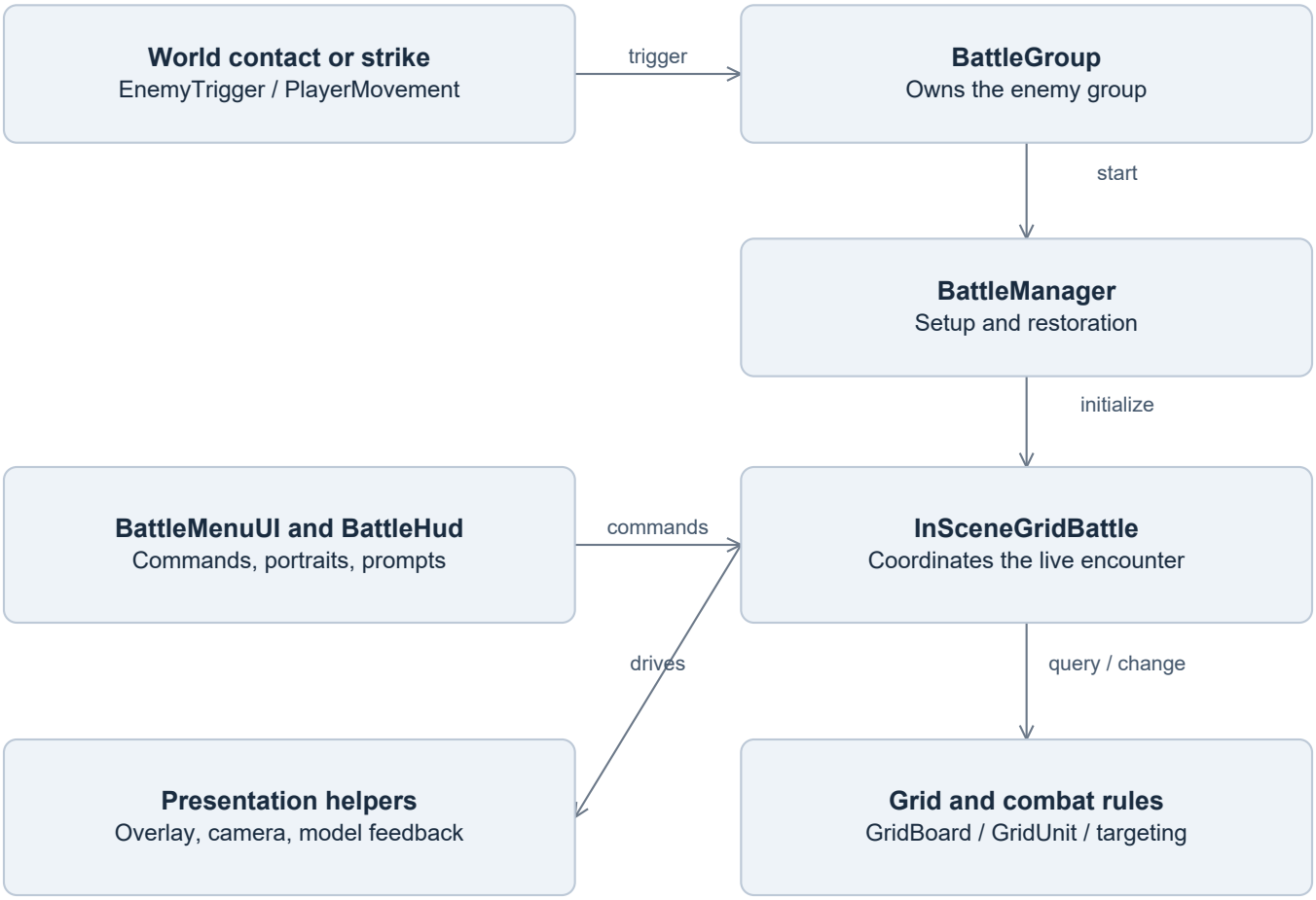
This guide describes the files currently in this project. It separates implemented behavior, prototype tuning, and future proposals. The placeholder roles are not final story characters or approved character kits.

Read this to understand	Pages
How scripts connect and who owns the data	2-3
Shared attacks, unique skills, and adding future mechanics	4-5
Battle entry, turns, reserves, and movement	6-8
One attack from button press to damage	9-10
Reactive defense and cinematic QTEs	11
Persistence and the overworld	12
Current requirements and unfinished systems	13
Script directory and reading exercises	14-16

Scope: first-party scripts under Assets/Scripts, relevant prefabs and SampleScene wiring, assembly boundaries, and focused test source. Third-party packages, imported art, and generated Unity files are outside this architecture walkthrough. Unity version in ProjectSettings is 6000.4.4f1. No gameplay or test run was performed for this documentation task.

Source paths in the code trails are relative to D:/Repos/Turn-Based RPG Game. Line numbers identify this snapshot and may move as the project evolves.

02 The scripts communicate in layers



The arrows show the main direction of calls. The UI also reads the encounter and listens for notifications; the encounter directly drives camera, overlay, and feedback helpers. These are ordinary C# references and method calls, not network messages or a global event bus.

Three concrete connections

Direct call: BattleGroup.TriggerBattle calls BattleManager.TryStartBattle. The manager creates an InSceneGridBattle and passes it the grid, actor transforms, starting cells, camera, and a completion callback.

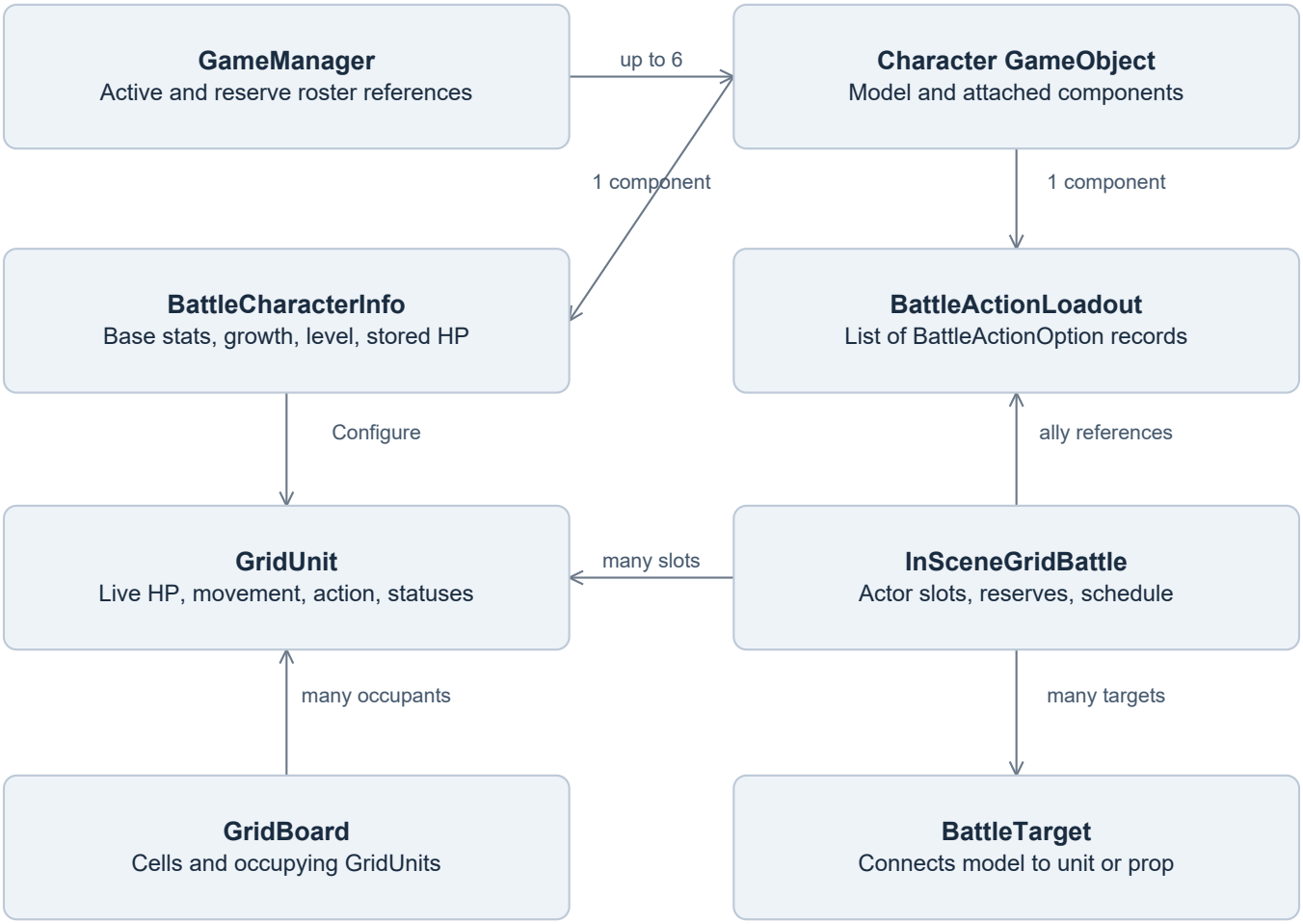
Event subscription: BattleMenuUI.Bind subscribes to ActionResolved and TurnStarted to rebuild root commands, and PartyChanged to refresh portraits. Unbind removes those subscriptions.

Callback: the encounter calls its finished callback with victory true or false. BattleManager.CompleteBattle restores the overworld and handles rewards. GridBoard also accepts a TraversalFilter callback so terrain physics can veto a move.

Code trail
Assets/Scripts/UI/BattleMenuUI.cs:123 (public void Bind)
Assets/Scripts/Battle/BattleManager.cs:159 (encounter.Initialize)
Assets/Scripts/GridPrototype/TerrainBattleGrid.cs:29 (Board.TraversalFilter = ClearEdge)

03 The runtime data relationship map

This is an ERD-style ownership map of in-memory objects. There is no relational combat database. “One to many” means an object holds a list or collection; it does not imply a database table.



A character GameObject also holds BattleCharacterInfo and DefensePrototypeSettings. At battle entry, Configure copies effective character stats into a new GridUnit. The loadout stays attached to the character; the menu reads the active actor’s loadout.

The encounter keeps corresponding lists of actors, units, and foot offsets. An actor index identifies a slot across those lists. InitiativeOrder schedules these indices without sorting the lists themselves. BattleTarget links a slot to its visible model for picking, hover, HP display, and hit resolution.

Lifetime matters: the GameObject and its loadout can outlive many battles. GridUnit, GridBoard, targeting state, and the Initiative schedule belong to one encounter. A benched GridUnit remains alive in the reserve list during that fight.

Code trail
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:14 (private readonly List<Transform> actors)
Assets/Scripts/GridPrototype/BattleCharacterInfo.cs:45 (public void Configure)
Assets/Scripts/GridPrototype/InSceneGridBattle.Party.cs:9 (public sealed class ReserveMember)

04 Shared attacks and unique skills today

Characters do not have separate subclasses such as VanguardCombat and ArcanistCombat. They use **composition**: the same components attached to different GameObjects, with different field values and action records. A prefab is the reusable template for a GameObject; an Inspector component is a script instance attached to it.

Prefab	Authored action examples	Runtime additions
Vanguard	Heavy Slash, Ward, Push, Sweeping Blow	Quick Strike
Arcanist	Arcane Bolt, Ember Volley, Renew, Ward	Push, Strike, Quick Strike
Balanced	Strike, Piercing Attack, area/multi-target skills, six QTE examples, magic, Blast Flask, Push	Already has two damage attacks
Sentinel / Skirmisher / Mystic	Heavy Slash, Ward, Push, Sweeping Blow	Quick Strike

This table reflects serialized prefab lists plus EnsureUniversalAttacks at battle entry. Scene instance overrides can change prefab values. The reserves currently reuse a placeholder list; different names and stats do not automatically mean distinct mechanics.

What is shared and what is separate

Shared code: Ward on Vanguard and Ward on Arcanist run through the same target selection and ApplyHit logic. A unique action can use that same code with a different pattern, power, scaling stat, element, or status.

Separate data: each loadout owns its own BattleActionOption records. Ward is duplicated in both prefab lists; it is not a reference to one central Ward asset. Editing Vanguard’s Ward does not automatically update Arcanist’s Ward. Names are display text, not a global ability ID.

Universal fallback: EnsureUniversalAttacks checks for any Push effect and counts damage actions in the Attack category. It fills missing basic attacks. It does not require every character to have an identically named Strike. For Vanguard, Heavy Slash counts as the first basic attack; Quick Strike becomes the second.

Category chooses the menu bucket: Attack, Skill, Item, or Magic. It does not determine damage type, scaling, targeting, or an MP cost. Those are separate fields; MP is not implemented.

Code trail
Assets/Scripts/GridPrototype/BattleActionLoadout.cs:104 (public void EnsureUniversalAttacks)
Assets/Prefabs/Player/Prototype Vanguard.prefab:348 (- Name: Heavy Slash)
Assets/Prefabs/Player/Prototype Arcanist.prefab:209 (- Name: Arcane Bolt)

05 How to extend a character without duplicating combat

A variation that the existing data already supports

For an additional ranged magical attack, author another BattleActionOption on the intended character's loadout. Set Name, Category, Reach, Pattern, DamageType, Scaling, StatScaling, and any supported status/QTE fields. The existing menu builds its row and the existing resolver executes it. No new character combat class is needed.

For a skill available to two characters, put an option in both lists today. Keep their shared settings aligned manually. To make a skill exclusive, author it only on the intended character. ChooseAction checks that the option belongs to the active character's loadout; it does not offer every option from every party member.

A genuinely new mechanic requires new rules

A new damage number is data. Debt, a new resource, delayed repayment, a new status lifecycle, or an action that alters turn scheduling is new behavior. These cannot be implemented faithfully by merely renaming Strike. The behavior needs an agreed design, state ownership, lifecycle hooks, UI feedback, and focused tests.

For a future Debt kit, first decide when debt becomes due, what happens on skipped turns, whether it follows the character into reserve, and how repayment interacts with defeat. Then add the rules in a small testable model and connect it through the encounter. This is a development recipe, not approval of a particular Debt design.

Recommended future separation of shared definitions

Current implementation	Possible future architecture
Each loadout embeds full option records.	Reusable AbilityDefinition assets hold shared immutable rules.
Quantity and FavoriteSlot live in the same option as damage and targeting.	Per-character state holds item quantity, favorites, ownership and future unlocks.
One character's edit can diverge from another's copy.	Both characters reference the same definition; explicit overrides remain deliberate.

In Unity, ScriptableObject assets would be one reasonable way to implement that library. They are **not implemented here**. Never store each character's mutable item count on a shared ability asset, or consuming an item for one character could change it for everyone.

Code trail
Assets/Scripts/GridPrototype/BattleActionLoadout.cs:11 (public sealed class BattleActionOption)
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:412 (public bool ChooseAction)

06 Entering battle on the existing terrain

Stage	What the code does
1 Detect	EnemyTrigger handles possessed-player contact. PlayerMovement can instead use WorldEncounterStrike to find an enemy in front and request a pre-emptive encounter.
2 Gather	BattleGroup passes its spawned enemies to BattleManager. GameManager supplies the active party, with the controlled member first, plus reserves.
3 Validate	BattleFormation sizes the region around participants. TerrainBattleGrid samples real colliders. TryPlan finds unique, nearby supported cells and checks entry paths before the world is suspended.
4 Suspend	BattleManager snapshots transforms, activation, colliders and rigidbodies; pauses relevant movement/AI; switches input; disables the original camera.
5 Build	The temporary encounter creates units, targets, overlay, copied battle camera, loadout references, and Initiative schedule.
6 Present	The entry partial eases actors and camera into battle, fades the grid/HUD, and only then starts the opening turn.

The manager currently requests PreserveWorldPositions. Characters snap to nearby legal cells around their actual overworld positions; companions do not always teleport into a fixed triangle. Formation-marker and older formation helpers still exist, but they are not the active placement policy for this call.

The base grid dimensions are 14 by 12 with 2-metre cells. Bounds can expand to include participants, with a 64-cell limit per axis in TryGridBounds. An overly spread group or an unsafe starting cell causes setup to fail before the suspension stage.

The original ground remains visible and collidable. This is a temporary tactical interpretation of the scene, not a scene load and not a replacement landscape. Partial setup exceptions use the manager’s restoration path.

Code trail
Assets/Scripts/Battle/BattleManager.cs:80 (public bool TryStartBattle)
Assets/Scripts/GridPrototype/BattleEntryPlan.cs:18 (public static bool TryGridBounds)
Assets/Scripts/GridPrototype/InSceneGridBattle.Entry.cs:52 (public void AdvanceEntry)

07 Turns and reserves

Turn order: InitiativeOrder sorts descending effective Initiative, using stable actor index for ties. Every living actor receives one scheduled turn per round. Pre-emption moves the controlled actor's existing turn to the front of round one; normal order returns in round two.

Moment	State change
BeginTurn	Tick start effects. If alive, add 3 movement to the saved balance, restore the action, clear Block, and open the turn.
Move	Spend movement only. The action remains available. A legal move may happen before or after an action.
Attack / skill / item	Spend the main action once at commit. Items also consume one quantity. The turn stays open.
Block or Dash	Spend the action. Block enables mitigation; Dash grants 3 additional movement in the current prototype.
EndTurn	Tick eligible end effects, close action access, retain unused movement, and advance the schedule.

Example: start with 0 movement, gain 3, walk 2 cells, attack, then end with 1 saved. Your next normal turn begins with 4. A diagonal adjacent step still costs 1. An action does not secretly end the turn.

The enemy loop is simpler than the player action system

EnemyRound advances through scheduled enemies until an ally is next. Each enemy picks the nearest living ally, approaches legal melee reach, then performs a single hit or three-hit demonstration combo. It does not choose from a BattleActionLoadout. The method name EnemyRound does not mean all allies always act before all enemies.

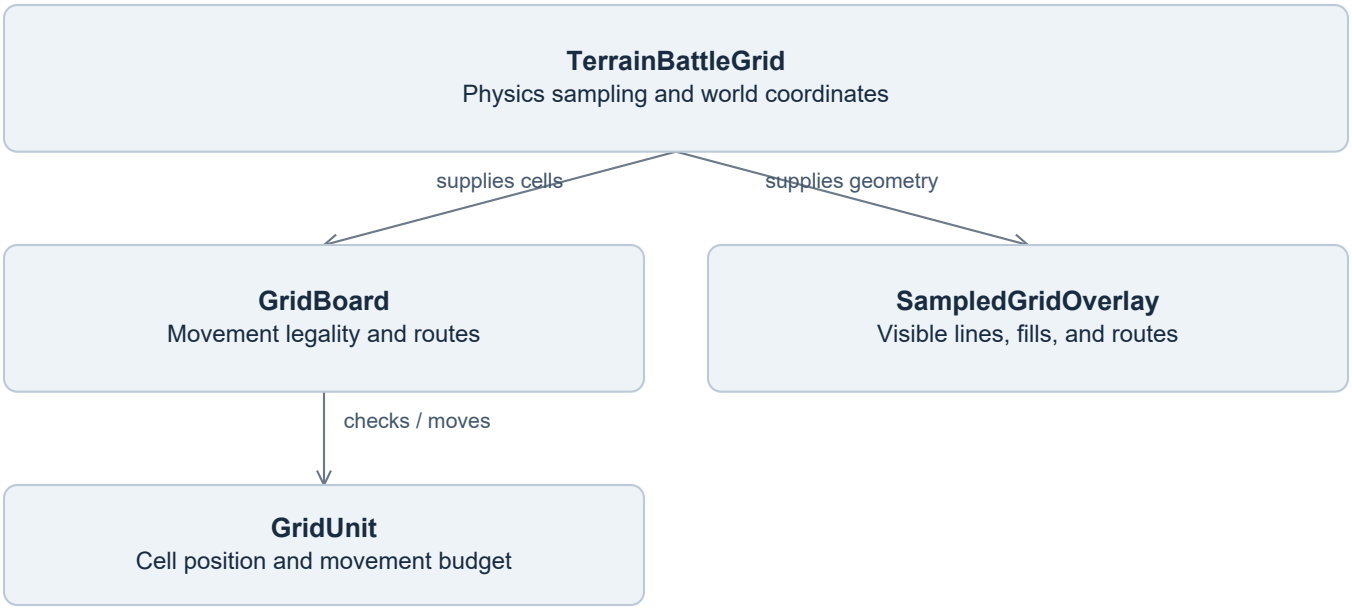
A tactical swap replaces the current slot

SwapWithReserve spends the outgoing member's action and transfers its remaining movement. The incoming member gets no new action, movement grant, or start tick. Its old personal movement bank is deferred until its next normal turn. The current round keeps its schedule; the replacement's Initiative affects the next round.

Living reserves deploy automatically only after all deployed allies fall. If none can take over, the encounter reports defeat. Tactical swaps do not rewrite GameManager's default roster.

Code trail
Assets/Scripts/GridPrototype/GridBattleModel.cs:49 (public void BeginTurn)
Assets/Scripts/GridPrototype/GridBattleModel.cs:36 (public bool HandOffTurnTo)
Assets/Scripts/GridPrototype/InitiativeOrder.cs:30 (public int Next)
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:817 (private IEnumerator EnemyRound)

08 The battle grid has three separate jobs



Terrain sampling: rays inspect the center and corners of each cell. Slope, supported surface, body clearance, water layer, and intermediate route clearance affect legality. The grid stores one surface per X/Z coordinate; stacked bridge/cave layers are not a supported navigation model.

Movement rules: GridBoard.CanTraverse checks destination occupancy, height difference, eight-direction alignment, diagonal corner clearance, and jump range. Long edges are jumps across water, not shortcuts through ordinary terrain or units. JumpRange defaults to 2 and JumpHeight to 1; Strength does not yet calculate JumpHeight.

Path search: Reachable explores legal edges within a budget, records the cheapest cost and parent for each cell, and reconstructs GridRoute stops. Equal-cost routes prefer shorter geometric length, reducing needless zigzags. This is a custom weighted search, not Unity NavMesh pathfinding.

A one-cell water gap takes a two-cell jump costing 2 movement. Saving 20 movement does not increase a character's jump range: a two-cell gap still needs range 3. Height and horizontal range are separate checks.

Movement commit: TryMove spends the cost and changes GridUnit.Position first. InSceneGridBattle.Move then animates the model along the returned route while interactions are locked. The visual model catches up to an already committed logical position.

The standalone GridPrototypeArena shares GridBoard but creates its own test terrain through GridTerrainView. SampleScene uses sampled real terrain and SampledGridOverlay. Changing one arena's presentation does not automatically change the other.

Code trail

Assets/Scripts/GridPrototype/GridBattleModel.cs:114 (public bool CanTraverse)
Assets/Scripts/GridPrototype/GridBattleModel.cs:149 (public Dictionary<Vector2Int, GridRoute> Reachable)
Assets/Scripts/GridPrototype/TerrainBattleGrid.cs:67 (private void Sample)

09 Follow one attack from selection to result

Step	Concrete call and responsibility
Choose	BattleMenuUI offers options from battle.Loadout. ChooseAction validates ownership, availability, and current action readiness.
Aim	ToggleTarget records cells. GridTargeting.CanTarget and Footprint evaluate reach, pattern, obstruction and sight. Multiple-hit actions store repeated cells in selection order.
Approach	TryActionApproach finds the cheapest affordable origin from which every selected cell can be targeted. No legal common origin means rejection.
Commit	TryPrepareAction rechecks readiness and approach, commits needed movement, builds target batches, spends one action, and decrements one item if applicable.
Present	The returned coroutine animates any approach. Signature actions run ResolveSignature and PressQte before the hit sequence.
Resolve	ResolveHits loops over target batches. Unit hits use BattleActionOption.ApplyHit. Props use a separate damage/Push branch.
Refresh	ActionResolved refreshes commands. Defeated targets leave occupancy. ResolveOutcome checks the teams. A surviving actor can still move and explicitly end its turn.

Targeting is cell-based. Empty cells are legal and still spend the action/item. Area and piercing footprints can hit allies; the resolver does not apply an automatic enemies-only filter. A multi-hit action can assign several hits to the same cell.

The preview and actual damage use the same footprint helper so their shapes agree. Sight combines conservative grid obstruction rules with scene raycasts. Reach is Chebyshev distance: the larger of horizontal and vertical cell offsets. This is not a finalized distance/elevation accuracy formula.

Important implementation detail: commit builds batches of BattleTarget references before the presentation runs. The scene is not a real-time free-for-all during that animation. New designs that move targets mid-attack would need a deliberate decision about when to retarget or revalidate.

Code trail

Assets/Scripts/GridPrototype/InSceneGridBattle.cs:481 (public bool TryPrepareAction)
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:575 (private IEnumerator ResolveHits)
Assets/Scripts/GridPrototype/InSceneGridBattle.Approach.cs:9 (public bool TryActionApproach)
Assets/Scripts/GridPrototype/BattleActionLoadout.cs:227 (public static HashSet<Vector2Int> Footprint)

10 Damage and statuses are shared rules

BattleActionOption.ApplyHit first rejects dead targets and rolls HitChance. A missed hit returns immediately. A landed damage action calls CombatMath.Damage, then GridUnit.TakeDamage. It subsequently rolls StatusChance and grants any authored Barrier. Each hit on each target rolls separately.

A real data example with explicit assumptions

Vanguard’s prefab Heavy Slash has base damage 12 and Strength scaling 1. With its authored base Strength 14, no added growth or allocated points, and an illustrative target Physical Defense of 20, raw damage is 26. Matching defense gives round(26 x 100 / 120) = 22. Effective Block halves that integer to 11. With 6 Barrier, TakeDamage removes the shield and takes 5 HP.

Those numbers demonstrate the current prototype formula; they are not final balance. DamageType selects Physical or Magical defense. Scaling independently chooses Strength or Magic. Element is separate metadata: Fire does not automatically burn, and there is no elemental resistance formula yet.

Effect	Current lifecycle
Burning	At the afflicted unit’s turn start, deal its configured tick damage and remove one stack. Stacks mean remaining ticks; they do not multiply tick damage.
Regeneration	At turn start, heal the configured amount and remove one stack. Burning runs first; this healing does not revive a unit killed by Burning.
Paralyzed	Prevents dodge, parry and Block; cancels existing Block. Does not remove movement/action or skip the turn. End-turn decay requires it to have existed at turn start.
Barrier	Separate absorption pool before HP, including periodic damage. Depletes with damage; otherwise lasts for the encounter. Not an enum status stack.

Status application adds the entire authored stack amount, capped at 10. Ember Volley uses three hit slots with a separate 50% chance of two Burning stacks on each landed hit. A shield absorbing all HP damage does not convert that hit into a miss.

Push takes a separate branch. It requires strictly greater Strength, a clear level land destination, and moves one cell. Invalid pushes still consume the committed action. Chilled, Weakened, Exposed, general cleansing, and skip-turn/debt rules are not implemented.

Code trail
Assets/Scripts/GridPrototype/BattleActionLoadout.cs:45 (public bool ApplyHit)
Assets/Scripts/GridPrototype/CombatEffects.cs:74 (public static int Damage)
Assets/Scripts/GridPrototype/CombatEffects.cs:50 (void TickStartEffects)

11 Reactive defense and QTEs use separate clocks

Enemy attacks use DefenseImpact

EnemyAttack faces the actors and waits for camera framing before beginning a wind-up. Each impact gets a new DefenseImpact. The encounter supplies elapsed time and fresh dodge/parry input; the rule returns success, too early, too late, conflicting input, or another failure result.

The first press commits for that hit. AdvanceDefense evaluates input before advancing the clock, against the previously displayed timing position. Resolve is idempotent: the same impact cannot apply damage twice. Successful dodge/parry prevents damage and enemy on-hit status; failed defense still benefits from effective Block.

A three-hit combo creates three independent opportunities. Missing hit one does not remove the next two chances. Parrying every hit earns one reachable counterattack. Logical grid occupancy stays unchanged; reaction poses and sparks are presentation.

DefensePrototypeSettings lives on allied actors and currently supplies even the demonstration enemy damage/wind-ups for the targeted ally. That is prototype coupling, not a finished enemy attack authoring system. Latency compensation defaults to zero; assistance and timings remain tunable.

Signature actions use PressQte

PressQte supports SinglePress, FixedSequence, RepeatedTap, Hold, Staged, and DualInput. These are deadline or charge challenges, distinct from the narrow dodge/parry window. BattleActionOption holds the chosen mode, inputs, duration, and bonus.

ResolveSignature starts the QTE after framing and preparation, waits for its result, then passes base power plus the success bonus to ResolveHits. Failure keeps base power. The subsequent hit/scaling/defense rules still run: QTE success is not an unconditional HP subtraction.

BattleHud draws the prompts and countdown; QteLayout places them, ControllerGlyphs supplies device artwork, and QteProgressRing / QteTriangleCue draw feedback. The HUD reads the timing model instead of owning a competing timer.

Input ownership: readiness flags prevent regular movement/menu commands during entry, attack presentation, reactions, inspection, and swap selection. Relevant timing updates pause while the app is unfocused. These are scripted camera/model sequences, not finished authored cinematics.

Code trail

Assets/Scripts/GridPrototype/DefenseImpact.cs:51 (public bool Attempt()
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:669 (public void AdvanceDefense)
Assets/Scripts/GridPrototype/InSceneGridBattle.cs:523 (private IEnumerator ResolveSignature)
Assets/Scripts/GridPrototype/PressQte.cs:6 (public enum QteMode)

12 What survives a battle and what is rebuilt

Data	Owner and lifetime
HP	GridUnit during battle; copied into BattleCharacterInfo on exit. Next battle restores HP with knocked members clamped to at least 1.
Movement, action, status stacks, Barrier	Encounter GridUnit. Preserved while benched within that fight; rebuilt next encounter.
Item quantities and favorites	BattleActionLoadout on the character. Persist across encounters in the running session.
XP, levels and allocated points	PartyProgression member records and BattleCharacterInfo. Deployed participants get full victory XP; unused reserves half, with fractional remainder carried.
Default active roster	GameManager.BattleParty / ReserveParty. Battle swaps are temporary; out-of-combat party selection edits these defaults.
World position and physics state	BattleManager ActorSnapshot restores participants on exit. Victory removes the triggering group’s enemies.
World props	Destruction and moved prop state are not covered by actor snapshots. Place effects belong to WorldPlaceService, independent of a temporary combat unit.

These are session lifetimes, not a disk save system. Do not assume Serializable means progress is saved automatically: it lets Unity represent authored fields, but runtime changes still need an explicit save/load design.

How the overworld connects

PlayerInputReader supplies world/battle input maps. PlayerMovement and PlayerRotation control the possessed character; PartyFollower handles companions. GameManager changes possession and follower ownership. BattleManager temporarily suspends those behaviors rather than using them for tactical moves.

WorldManipulationController is a separate overworld input/UI driver. It queries PlaceWorldAbility on the controlled actor and asks WorldPlaceService to validate restoration, dragging, relocation, and dismissal. WorldPlaceObject stores authored forms and origin state; WorldPlaceSocket can unlock a shortcut.

Place changes actual world geometry, so a later TerrainBattleGrid samples the resulting colliders. The object tools are disabled during battle. This does not make Place a BattleActionOption or implement its future combat kit.

Code trail
Assets/Scripts/Battle/BattleManager.cs:177 (private void CompleteBattle)
Assets/Scripts/GridPrototype/InSceneGridBattle.Party.cs:93 (public void StorePartyHealth)
Assets/Scripts/GridPrototype/PartyProgression.cs:61 (public int AwardExperience(int amount, IReadOnlyCollection)
Assets/Scripts/GridPrototype/World/WorldPlaceService.cs:63 (public bool TryRestore)

13 Product requirements snapshot

Purpose: a 3D party RPG combining deliberate grid positioning, independent character turns, and readable reactive defense. This is a current-state PRD summary, not approval of additional mechanics. “Implemented” below means present in code; it does not mean visually or experientially verified in this task.

Requirement	Current implementation and boundary
Party identity	Three active members and up to three reserves, individual stats/loadouts, tactical swap and individual XP. Final identities and unique kits remain open.
Position matters	Eight-direction grid movement on existing terrain; costs and carryover separate from actions. Strength-driven jump thresholds and layered navigation remain unfinished.
One turn per round	Initiative order, stable ties, first-round pre-emption, explicit End Turn. Turn-skipping effects and Debt interactions remain to be designed.
Actions are reusable	Per-character option lists, patterns, scaling, hit/status chances, items and QTEs. No central ability asset library, learning/unlock system, MP, or complete inventory.
Defend each hit	Dodge/parry per impact, Block safety net, combo counter. Current enemy routine is single-target melee; full party AoE reactive encounters remain future work.
Combat effects	Burning, Regeneration, Paralyzed, Barrier and limited Push. Chilled/Weakened/Exposed, equipment modifiers and elemental resistances are incomplete.
Growth and persistence	Prototype automatic growth, allocated stats, victory XP and session HP. No disk saves, checkpoints, respec, or finished progression economy.
Equipment baseline	One weapon, one outfit, two accessories; outside-combat changes and no carry weight are design direction. A functioning equipment pipeline is not implemented here.

Useful acceptance scenarios for future work

A new shared skill should behave consistently for two owners while keeping their resources separate. A unique skill must not appear on unrelated members. A movement command must never accidentally spend the main action. A rejected target must preserve resources. An accepted action must spend its cost once even across QTE feedback and multiple hits.

A new effect should define its creation, tick, expiry, reserve, defeat, and encounter-exit behavior. A new input feature should declare when it owns input and when it releases it. These are review criteria for future changes, not new implemented systems.

Code trail
AGENTS.md:10 (## Confirmed direction)
AGENTS.md:415 (## Six-member roster and reserve rules implemented)
Assets/Scripts/GridPrototype/CombatEffects.cs:7 (public enum CombatStatus)

14 Script atlas for the battle core

Folder root: Assets/Scripts. Related classes can share a file. “Partial” files are pieces of one compiled class, not separate components that send messages to one another.

File or group	Responsibility
Battle/BattleGroup.cs	Spawned enemy group, encounter request, cooldown, once-only XP reward.
Battle/BattleManager.cs	World-to-battle handoff and ActorSnapshot restoration.
Battle/BattleState.cs	Unused enum sketch. It is not the state machine driving this encounter.
GridPrototype/InSceneGridBattle.cs	Live coordinator: input, targets, actions, effects, enemy routine, turns, outcome.
InSceneGridBattle.Entry.cs	Entry camera, actor transitions, grid fade and opening turn.
InSceneGridBattle.Approach.cs	Find a movement destination that makes all chosen targets legal.
InSceneGridBattle.Party.cs	Reserve GridUnits, tactical replacement, takeover, HP storage.
GridBattleModel.cs	GridCell, GridUnit, GridRoute and GridBoard: budgets, occupancy, traversal, route search.
CombatEffects.cs	The other part of GridUnit, status lifecycle, Barrier and CombatMath.
BattleActionLoadout.cs	BattleActionOption data, defaults, favorites, universal attacks, GridTargeting.
BattleCharacterInfo.cs / PartyProgression.cs	Authored/effective stats and stored HP; member XP and level rewards.
InitiativeOrder.cs / BattleEntryPlan.cs	Stable slot scheduling; entry plan and BattleFormation helpers.
TerrainBattleGrid.cs / SampledGridOverlay.cs	Sample world terrain and clearance; draw the tactical overlay.
BattleTarget.cs / BattleHoverInfo.cs	Unit/prop adapter and hover focus; optional presentation metadata.
BattleDestructible.cs / BattleTargetHighlight.cs	Prop health and push properties; target outline geometry.
DefenseImpact.cs / DefensePrototypeSettings.cs	Per-impact timing rule; editable prototype parameters.
DefenseVisual.cs / DefenseSlowMotion.cs / ParryBurst.cs	Model poses, local recovery time scaling, and spark feedback.
PressQte.cs / QteLayout.cs	QTE state/input rules; prompt anchor positions.
GridCameraController.cs	Tactical free flight, focus, follow, orbit and defense framing.
GridPrototypeArena.cs / GridTerrainView.cs	Separate sandbox driver and its procedural continuous terrain.

Unless a folder is shown, files above are in GridPrototype. That historical folder name now includes core systems used by SampleScene. It no longer means “used only in the old sandbox.”

15 Script atlas for the surrounding game

File or group under Assets/Scripts	Responsibility
GameManager.cs	Possession, active/reserve rosters, progression registration, followers.
GameManager.Switching.cs / .PartySelection.cs	Overworld possession transition; changing default active members.
Player/PlayerInputReader.cs	Input maps, action values and press edges.
Player/PlayerMovement.cs / PlayerRotation.cs	World locomotion/strike and world camera rotation/follow.
Player/PartyFollower.cs / WorldEncounterStrike.cs	Companion following; nearby frontal encounter target query.
Enemy/EnemyMovement.cs / EnemyTrigger.cs	World roaming/chasing/awareness; contact encounter trigger.
UI/BattleMenuUI.cs / BattleMenuButton.cs	Commands, categories, favorite/list interaction; button visuals/icons.
UI/BattleHud.cs	Turn portraits, examination, target details, defense and QTE display.
UI/BattlePortrait.cs	Portrait clicks plus BattlePortraitCapture model rendering helper.
UI/BattleControllerNavigation.cs	Virtual cursor, panel navigation and controller camera routing.
GridPrototype/BattleNavigationState.cs	Small controller mode/area/deadzone state model.
UI/BattleSwapPanel.cs / ProgressionMenu.cs	Reserve confirmation; outside-battle stat allocation and roster UI.
UI/ControllerGlyphs.cs	Shared keyboard/controller sprite catalogue and device-family styling.
UI/QteProgressRing.cs / QteTriangleCue.cs	Specialized prompt charge and tactile cue graphics.
World/WorldManipulationController.cs	Owns overworld object-tool input, picking, preview and panel.
GridPrototype/World/PlaceWorldAbility.cs	Character capability marker and range tuning.
GridPrototype/World/WorldPlaceService.cs	Restore/move/return validation and active Place effect ownership.
GridPrototype/World/WorldPlaceObject.cs / WorldPlaceSocket.cs	Authored object forms/origin and socket unlock behavior.

Editor builders and assembly boundary

Assets/Editor/GridPrototype/GridPrototypeBuilder.cs builds the standalone test scene. Assets/Editor/WorldPlacePrototypeBuilder.cs builds the Place test area. They are editor authoring tools, not the live turn loop.

GridPrototype/Game.Grid.asmdef groups its subtree into the Game.Grid assembly, referencing Unity.InputSystem. World and UI scripts outside that subtree can use it; the grid assembly avoids directly depending on GameManager or BattleMenuUI. Callbacks and public encounter APIs bridge that boundary.

Assets/Tests/GridPrototype/EditMode contains the Game.Grid.Tests editor-only assembly and focused regression sources. Tests are executable examples of intended behavior; they are not gameplay scripts.

16 Reading route and future change guide

If you want to change	Start here
What one character can use	Its prefab BattleActionLoadout.Options; inspect scene overrides too.
How damage or a status works	CombatEffects.cs, ApplyHit, then both player and enemy hit paths.
Movement cost, corners or jumping	GridBattleModel.cs; TerrainBattleGrid for physical clearance.
Turn order or resource refresh	InitiativeOrder plus GridUnit.BeginTurn/EndTurn and AdvanceTurn.
A targeting shape	GridTargeting.CanTarget / Footprint, preview and commit tests.
Presentation without rule changes	BattleHud, GridCameraController, DefenseVisual or QTE helpers.
Persistent inventory or a shared skill library	First separate definitions from character-owned mutable state.
A truly unique character resource	Agree its lifecycle, add a testable rule model, then integrate at encounter boundaries.

Three short code-reading exercises

1. Trace Heavy Slash. Open Vanguard’s prefab option, then ChooseAction, TryPrepareAction, ResolveHits, ApplyHit, CombatMath.Damage and TakeDamage. Identify where the action is spent versus where HP changes.

2. Trace one movement. Follow Confirm to GridBoard.TryMove, Reachable and CanTraverse, then back to Move. Notice the separate cell position and model transform. Change no code; predict which layer would reject a blocked diagonal.

3. Trace a reserve swap. Read SwapWithReserve, HandOffTurnTo and ReplaceSlot. Track the outgoing balance, incoming deferred balance, action flag, loadout reference and next-round Initiative value.

What the existing split does well and where it needs work

The small rule models accept explicit values and time, so tests can exercise movement, timing, effects and order without a full live game. Shared targeting avoids separate preview and damage shapes. Actor snapshots make the world/battle handoff reversible.

The split is incomplete. InSceneGridBattle.cs is about 910 lines and still coordinates many concerns. BattleHud and BattleMenuUI also combine substantial UI behavior. Action definitions mix mutable state with reusable rules; enemy attacks bypass the player option pipeline. These are sensible future refactoring candidates, not changes made by this guide.

Verification: this document was checked against source and prefab data. Relevant examples include PartyBattleTests (independent loadouts, swaps and approach), GridBattleTests, TerrainBattleGridTests, InitiativeTests, CombatEffectsTests, BattleActionTests, DefenseImpactTests and PressQteTests. Existing test source was inspected; tests and Play Mode were not run for this task.